

Physical Validation of Energy-Harvesting-Aware Path Planning: Lightweight Onboard Reinforcement Learning and Mapping on a TurtleBot 3 Burger

Lars De Leeuw

Internship Research Report

Host Institution: University of Antwerp

Study Program: M.Sc. Computer Science

Email: deleeuw@lars@icloud.com

Abstract—Ambiently-powered mobile robots must plan paths that balance travel cost against the energy they can harvest from a spatially varying environment. A recent multi-agent reinforcement-learning method, H-CMARL-DE, reported up to a 240% improvement in energy-harvesting efficiency in simulation, but was never validated on hardware. This report presents such a validation. We deployed a pre-trained Proximal Policy Optimization (PPO) policy, an energy-aware A* (EH-A*), and a shortest-path A* on a modified TurtleBot 3 Burger across two indoor maps (10×10 and 15×15 grids) and two distinct compute modes (centralized and decentralized), using overhead ArUco localization and luminance-derived energy maps. Because of the small on-board solar panel and indoor conditions, harvested energy was magnitudes below consumption, so we report *relative* outcomes. Energy-aware planning consistently altered the chosen route, but the harvesting payoff was below measurement noise and net energy was dominated by locomotion. Specifically turn count, which even decided the EH-A*-versus-A* comparison. The clearest pro-learning result was inference cost: A* search time grew with map size while the PPO forward pass stayed flat, and on the Pi, the two crossed on the larger map. The learned specific policy used during the experiments, however, did not navigate well on hardware: zero genuine goal completions in 44 runs (it approached the goal but never stepped onto it unaided), consistent with its egocentric 10×10 observation and an out-of-distribution single-agent deployment of a ten-agent policy. We detail the testbed, the campaign, and the threats to validity that shape these conclusions.

Index Terms—energy harvesting awareness, path planning, deep reinforcement learning, proximal policy optimization, embedded inference, ROS 2, mobile robots, fiducial-marker localization.

I. INTRODUCTION

AUTONOMOUS mobile robots increasingly integrate ambient energy harvesting (EH) from solar panels, or wireless inductive charging, to sustain long-term operation without manual recharging [1], [2]. Such ambiently-powered platforms are attractive for persistent tasks such as environ-

mental monitoring, warehouse logistics, and disaster response, where downtime for recharging is costly or impossible.

Ambient energy, however, is rarely uniform. Illumination, wireless signal strength, and occlusion vary across space and fluctuate over time, so the amount of energy a robot can collect depends on *where* it goes and *when* it arrives. Path planning can therefore no longer optimise distance alone: a shortest-path planner ignores harvesting opportunities and may strand the robot, whereas a planner that greedily maximises harvested energy over-extends travel time. Effective ambiently-powered navigation is fundamentally a multi-objective problem that trades path length against harvested energy while still avoiding collisions [1].

Classical planners such as Dijkstra’s algorithm [3] and A* [4] are optimal in static environments but adapt poorly to dynamic conditions and inter-robot dependencies, and they extend awkwardly to multiple objectives. Reinforcement learning (RL) offers a complementary trade-off: it learns adaptive, generalizable behavior by trial and error and handles multiple objectives naturally, at the cost of high training time but fast inference once trained [5], [6]. These properties make RL a natural fit for energy-harvesting-aware navigation in dynamic environments.

Building on this insight, and on their earlier work on energy-aware scheduling for ambiently-powered robot swarms [7] Mokhtari *et al.* [1] proposed H-CMARL-DE, a hierarchically conditioned multi-agent RL scheme that centralises only meta-level parameters priority ordering, reward-shaping weights, and sub-goal settings - while keeping per-agent policy learning and execution decentralised. In simulation (OpenAI Gymnasium and a ROS/Gazebo case study) it achieved up to a 240% improvement in energy-harvesting efficiency over classical, non-EH-aware planning, with only a marginal increase in path length. Crucially, that study was validated entirely in simulation; its authors explicitly list *real-world hardware validation* as future work.

Moving such a method from simulation onto a real robot

Work carried out under the supervision of M. Mokhtari (University of Antwerp – imec, IDLab).

raises challenges the simulator hides. The trained policy must run in real time on resource-constrained onboard hardware. The obstacle and energy maps which the simulator supplies for free must instead be built online from sensors or other means; and the policy must close a real observe-think-act loop without the training harness. This report addresses that transition through two themes the source paper does not treat in depth: *lightweight onboard reinforcement learning* running a compact policy on a Raspberry Pi-class single-board computer by exporting it to ONNX [8], [9] and *online mapping* deriving obstacle and energy grids from an overhead camera using ArUco fiducial localization [10] and a luminance-based light-intensity proxy.

Concretely, this work contributes:

- A lightweight, embedded-friendly inference path (ONNX) for the trained PPO policy (96KB, ONNX runtime +20MB) on a Raspberry Pi 4b, with verified numerical parity to the original model.
- A fair, energy-aware A* baseline that shares the bridge’s wire protocol with the learned planner, selectable at launch without changing the orchestrator.
- An instrumented TurtleBot 3 testbed with overhead-camera localization, luminance-derived energy maps, and direct power measurement, enabling a real harvested-versus-consumed energy comparison.
- An experimental methodology together with the empirical finding that the policy requires stochastic action sampling to navigate reliably on hardware.

The remainder of the report reviews related work (Section II), states the problem and our implementation approach (Section III), details the simulation baseline and the physical experimental setup (Section IV), and concludes (Section V).

II. RELATED WORK

A. Path planning: from classical to learned

Table I contrasts representative planning families. Dijkstra’s algorithm [3] and A* [4] are optimal in static grids but offer no adaptability and only limited support for multiple objectives. Sampling-based planners such as dynamic RRT improve coverage of continuous, dynamic spaces but forfeit optimality guarantees. Reinforcement learning [5], [6], [11] trades a high training cost for fast runtime inference, native multi-objective handling, and generalization, which is why it is well suited to the dynamic, stochastic, multi-objective setting of energy-harvesting-aware navigation.

B. Energy-aware planning

A first body of work minimizes *consumed* energy. Energy-efficient A* variants with motion primitives, online energy-constrained coverage planners, and aerodynamic-cost planners for fixed-wing UAVs have all been proposed (surveyed in [1]). These methods reduce the cost of motion but do not model energy *intake* from the environment.

C. Energy-harvesting-aware planning

A second body of work maximizes *harvested* energy. As summarized in Table II, most prior approaches target aerial or marine vehicles and a single harvesting modality (typically solar), are evaluated only in simulation, and rarely combine collision avoidance with multi-robot coordination. A notable exception is Plonski *et al.* [2], who plan energy-efficient paths for a *real* solar-powered ground robot using Gaussian-process solar maps and dynamic programming, but for a single robot and a single energy source. The source paper [1] fills the ground-robot, multi-source, collision-aware, multi-robot cell of this landscape, but in ROS/Gazebo simulation only.

D. Tooling and the gap addressed here

The learned planners in this study build on PPO [6] (with A2C [11] as the principal alternative) implemented in Stable-Baselines3 [9] over the Gymnasium API [12]; the meta-level optimization in the source method uses Differential Evolution [13], and the classical ROS baseline uses the Open Motion Planning Library [14]. Two aspects relevant to physical deployment remain under-explored for this task: *embedded inference* of a learned EH-aware planner on a single-board computer, and *online perception*—building the obstacle and energy grids from sensors rather than from the simulator. We address the latter with fiducial-marker localization [10], which substitutes for on-robot SLAM [15] in our controlled overhead-camera arena. To our knowledge, no prior work physically deploys a *learned* energy-harvesting-aware planner on a real ground robot and benchmarks it head-to-head against a classical energy-aware planner.

III. PROBLEM STATEMENT AND IMPLEMENTATION APPROACH

A. Energy-harvesting-aware navigation

We adopt the grid formulation of [1]. A robot navigates a discrete $M \times N$ grid in discrete time using an 8-connected neighborhood, traveling from a start cell to a goal cell while avoiding static and dynamic obstacles and *maximizing the energy harvested* along the way. Dynamic obstacles follow a parametric Markov motion model, and a spatiotemporal energy field, generated by K moving sources with spatial smoothing, temporal persistence ρ and diffusion D , is normalized to $[0, 1]$ to represent harvestable energy per cell. Do note that due to physical constraints and for reproducibility purposes our experiment used only static obstacles and static light sources. The per-step reward combines a sparse goal reward, a collision penalty, a per-move time penalty, and an energy term $\eta EH(\cdot)$ whose weight η trades goal-reaching against harvesting. Maximising harvested energy per unit distance is the principal objective; Section IV defines the metrics precisely.

B. The source method, in brief

H-CMARL-DE [1] solves a *multi-robot* version of this problem with a two-level scheme. A centralized meta-level uses

TABLE I
COMPARISON OF PATH-PLANNING METHOD FAMILIES (AFTER [1]).

Aspect	Dijkstra	A*	Dynamic RRT	Reinforcement learning
Environment type	Static	Static	Dynamic, continuous	Dynamic, stochastic
Optimality	Optimal	Optimal	No guarantee	Depends on training
Multi-objective	No	Limited	Difficult	Easy
Scalability	Poor	Limited	Good	Good
Adaptability	No	No	Yes	Yes
Computational cost	High	Low	Moderate	Low runtime, high training
Robustness	Low	Medium	High	High
Generalization	No	No	No	Yes

TABLE II
REPRESENTATIVE ENERGY-HARVESTING-AWARE PATH-PLANNING APPROACHES (AFTER [1]). A CHECK (✓) DENOTES A SUPPORTED PROPERTY.

Work	Algorithm	Robot type	Collision av.	Harvesting	Multi-robot	Implementation
Huang <i>et al.</i>	RRT	Fixed-wing UAV	–	Solar	–	Simulation
Huang & Savkin	Dynamic RRT	Multirotor UAV	✓	Solar	✓	Simulation
Ni <i>et al.</i>	Soft Actor–Critic	Solar UAV	–	Solar	–	Simulation
Liao <i>et al.</i>	Optimized ACO	Marine USV	✓	Flow	–	Simulation
Plonski <i>et al.</i> [2]	Dynamic programming	Ground robot	–	Solar	–	Real robot
Source paper [1]	RL / A*	Ground robots	✓	Multi-source	✓	ROS/Gazebo
This work	RL / A*	Ground robot	✓	Light (solar)	– (single)	Physical robot

Differential Evolution [13] to tune coordination and reward-shaping parameters, while each robot runs a decentralized PPO policy [6] hierarchically conditioned on the committed trajectories of higher-priority robots; time-indexed reservations guarantee collision-free execution. We keep this description deliberately brief: the present work does not modify the method, but *reproduces a trained instance of it on hardware* and compares it against a classical energy-aware planner.

C. From simulation to a single physical robot

Our task is the single-robot slice of the problem: take one trained policy off the simulator and run it on a real TurtleBot 3 Burger, against an energy-aware A* baseline, while measuring real harvested and consumed energy. We run the experiment across two distinct compute placements:

- the robot runs the planner and all other parts of the software framework besides the external tracker and experiment orchestrator (decentralized mode),
- the robot runs only the diagnostic code and the minimum essentials needed to get the robot remotely controlled (centralized mode).

Because the policy was trained for ten robots, the nine absent robots are stacked onto the real robot’s own grid cell so they are invisible to the policy. This pragmatic device, that lets the unmodified ten-robot model drive one physical robot, was checked with the author of the source paper.

D. Implementation approach

Figure 1 shows the deployed architecture. A laptop *orchestrator* performs all perception and bookkeeping: it localizes the robot from an overhead camera, builds the obstacle and energy grids, and runs the observe-think-act loop. It communicates over a single TCP connection with a *bridge node*; everything

beyond the bridge is a ROS 2 node. For each step the orchestrator sends the current pose, heading and a `predict` request, the bridge forwards it to the active *planner node*, which returns one of eight discrete actions, and the bridge then drives the chosen adjacent cell through a *navigation node* that executes a rotate-then-drive motion using the IMU and ODOM sensors to minimize drift during locomotion.

Planner selection: The three planners are ROS nodes: a PPO node and an A* node that runs either the energy-aware or the shortest-path variant. The orchestrator only ever talks to the bridge and cannot tell which planner is active; the planner is chosen at *launch time*, not by switching network ports. This keeps the orchestrator and the wire protocol identical across the three campaign arms, so the comparison is fair.

Compute placement: The entire ROS 2 stack (bridge, navigation, planner) runs either on the robot’s Raspberry Pi 4B (*decentralized*) or on a desktop virtual machine (*centralized*); only sensor logging and the motor-command layer always stay on the robot. This *compute-placement* variable is what “centralized versus decentralized” means physically here, and it is distinct from the algorithmic centralization of the source method. Bringing the platform up was itself non-trivial: the robot was migrated from ROS 1 to a clean ROS 2 Humble install, and the desktop VM hosts a reproducible ROS 2 workspace reached over SSH, a sim-to-real engineering cost worth recording.

Lightweight onboard inference: The PPO node runs the exported policy. The reference path through Stable-Baselines3 and PyTorch [9] carries a ~ 2 GB dependency stack; exporting the network to ONNX and serving it with `numpy` and `onnxruntime` [8] reduces this to 15–25 MB with verified numerical parity (arg-max actions bit-identical over 200 samples). Section IV-E shows that this cheap forward pass is the

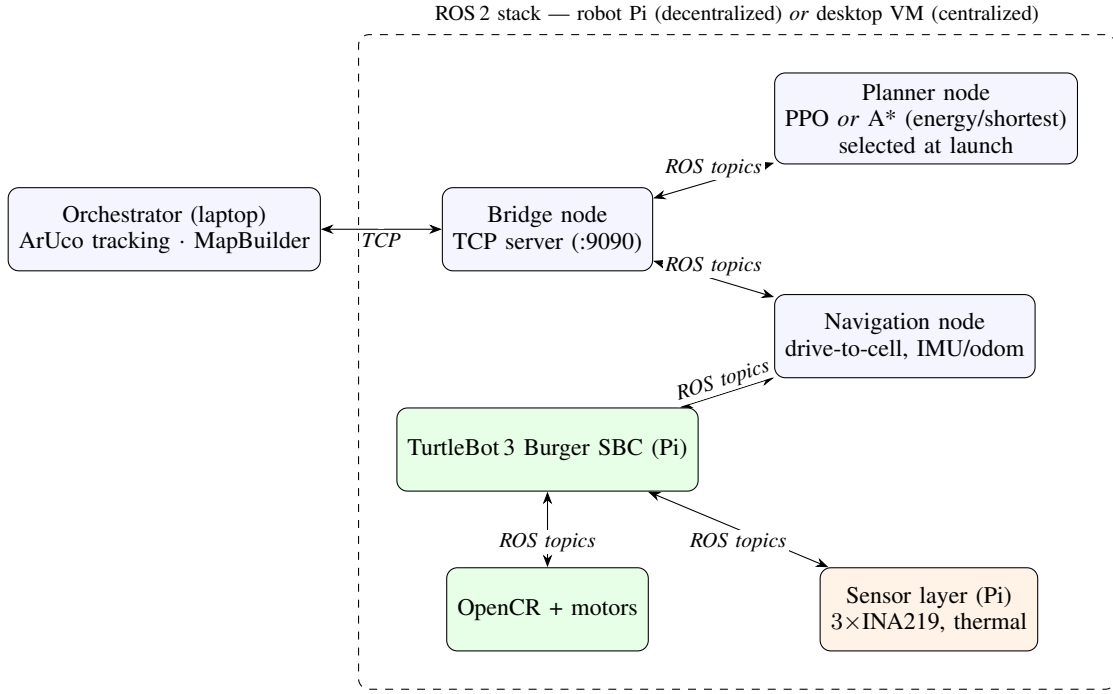


Fig. 1. Deployed architecture. The laptop orchestrator handles perception (overhead ArUco tracking, map building) and runs the observe-think-act loop (for experimental logging purposes), talking to the robot only through a bridge node over TCP. Everything beyond the bridge is a ROS 2 node: the planner (PPO or A*, selected at launch) and the navigation node. The whole ROS 2 stack runs on the robot’s Pi (decentralized) or on a desktop VM (centralized); the motor base and the power-sensor layer always stays on the robot.

policy’s clearest advantage on hardware.

Egocentric observation and its myopia: The policy observes a fixed 10×10 window centered on the robot, with out-of-grid cells zero-padded; this makes it grid-size-agnostic but *partially observable*. When the goal lies more than five cells away it is clamped to the window edge, so the policy sees only an approximate goal *direction*. In rollout testing this proved a major limitation: behind a thick wall or beyond the five-cell horizon the policy cannot perceive a detour and gets stuck. Maps therefore had to be designed around this myopia (thin walls, nearby light), and as Section IV-E reports, it is the leading explanation for the policy’s physical navigation failures.

Stochastic execution: The policy is run with stochastic action sampling over the soft-max of its logits rather than deterministic arg-max selection, which stalls against obstacles. Sampling occasionally proposes an illegal (blocked) move; the orchestrator blocks it and, after three consecutive illegal moves, records the run as a failure (Section IV-D).

Online mapping and localization: Perception is external to the robot by design. An overhead camera and ArUco markers [10] define the arena: four corner markers fix a floor plane that is virtually subdivided into grid cells, and a marker on the robot yields its cell and heading. Position is recovered from the *homography* of the marker center onto the floor plane (depth-independent and robust to the jitter of a small marker on a compressed video stream), while heading uses a planar-square PnP solution. For camera intrinsics a simple

heuristic was used, although more sophisticated calibration is fully supported as well. This external pose is pushed to the navigation node before every plan, deliberately avoiding the wheel-odometry and IMU drift that derailed early trial attempts, an external substitute for on-robot SLAM [15]. Meaning less energy has to be expended by the robot to obtain the mapping required.

Energy-aware A baseline:* The baseline is an 8-connected A* with an octile heuristic and two variants: a shortest-path mode and an energy-aware mode whose edge cost subtracts a weighted energy term, $cost = distance - \alpha \text{energy}(\text{cell})$, mirroring the energy-aware A* (CF-A*-EH) of the source paper. For our experiment we use $\alpha = 0.5$ to match the source paper.

IV. SIMULATION SETUP AND RESULTS

This section has two halves. Section IV-A-IV-B report the *source paper’s* simulation setup and results, which are the citable baseline this work attempts to reproduce. Section IV-C-IV-D describe the *physical testbed*, the devices, technologies, and experimental design that constitute this internship’s contribution. The results of the physical experiment are presented in (Section IV-E).

A. Source-paper simulation setup and hyperparameters

The source study [1] was implemented over the Gymnasium API [12] with PPO [6] and A2C [11] from Stable-Baselines3 [9]; its ROS/Gazebo case study used the Open Motion Planning Library [14] as the classical baseline, on a

TABLE III
ENVIRONMENT AND RL HYPERPARAMETERS OF THE SOURCE
SIMULATION [1].

Parameter	Value / range
Grid size	$10^2, 50^2, 100^2$
Goal reward R_{goal}	+50
Collision reward R_{coll}	-5
Move reward R_{move}	-1
Energy-reward weight η	3 (balanced)
Static obstacles	10% of cells
Dynamic obstacles	5
Obstacle speed $\ v\ $	train $\mathcal{U}(0, 2)$, test $\mathcal{U}(2, 3)$
Energy sources K	3
Persistence ρ	$\mathcal{U}(0.5, 0.95)$
Diffusion D	$\mathcal{U}(0, 0.1)$
Flow $\ u\ $	$\mathcal{U}(0, 0.5)$
Entropy coefficient	0.1
Learning rate	3×10^{-4}
n_{steps}	1000
Batch size	256
Discount γ	0.95
GAE λ	0.95
Value-function coefficient	0.3
A* energy weight α	0.5

TurtleBot 3 Burger model. Training used two million steps per configuration, a three-stage curriculum (navigation, then dynamic obstacles, then energy harvesting), and transfer learning to initialize larger grids; results were averaged over ten independent trials. Per-step inference was reported on the order of microseconds on an Intel Core i7 (12th generation), establishing real-time feasibility. Table III lists the environment and RL hyperparameters.

The trained policy used during our experiment (AIPPom10EH_continued.zip) reports slightly different values from Table III: entropy coefficient 0.08, batch size 500, GAE $\lambda = 0.91$, clip range 0.2, and 10 epochs over two million steps, while learning rate, γ , value coefficient and n_{steps} match.

Metrics: We reuse the source paper’s metrics [1]. *Success rate* is the fraction of attempts reaching the goal; *mission completion time* (MCT) is the mean duration of successful attempts; *average path length* (APL) and *average closest path length* (ACPL, the shortest-path reference) yield the path-length efficiency

$$\text{PL eff.} = \text{APL} / \text{ACPL} \quad (\text{closer to 1 is better}), \quad (1)$$

and *average energy harvested* (AEH) yields the key metric, energy-harvesting efficiency,

$$\text{EH eff.} = \text{AEH} / \text{APL} \quad (\text{harvested energy per unit length}). \quad (2)$$

B. Source-paper results

Across grid sizes, the learned planners run in real time while the heuristic planners require heavy precomputation (Table IV); H-CMARL-DE is about 34% faster than CTDE on average. In the ROS/Gazebo case study (Table VI) all methods reached 100% success, and H-CMARL-DE roughly doubled

TABLE IV
AVERAGE PER-STEP PLANNING RUNTIME BY GRID SIZE [1]. CFA* / CFA*EH ARE STATIC (PRECOMPUTED); CTDE AND H-CMARL-DE ARE REAL-TIME AND ADAPTIVE.

Method	10×10	50×50	100×100
CFA*	0.9 s	8.1 s	15.2 s
CFA*EH	1.6 s	11.3 s	17.4 s
CTDE	44 ms	428 ms	4.2 s
H-CMARL-DE	29 ms	302 ms	3.4 s

TABLE V
ENERGY-WEIGHT (η) ABLATION: PL AND EH EFFICIENCY FOR PPO AND A2C [1]. THE BALANCED OPERATING POINT $\eta = 3$ IS USED IN THE MAIN EXPERIMENTS.

η	PPO		A2C	
	PL eff.	EH eff.	PL eff.	EH eff.
0	1.02	0.19	1.08	0.18
1	1.18	0.18	1.10	0.17
2	1.16	0.23	1.22	0.21
3	1.05	0.27	1.17	0.22
5	2.78	0.41	2.50	0.35
10	2.95	0.49	2.70	0.42

the energy-harvesting efficiency of the classical ROS-OMPL planner (e.g. 0.29 vs. 0.13 in the four-robot layout), at a modest cost in mission completion time from deliberate energy-harvesting detours. Over the grid simulations the average improvement of H-CMARL-DE over the CTDE baseline was 7.3%. A one-way ANOVA across grid sizes found path-length efficiency to differ significantly ($p = 0.0168$) but energy-harvesting efficiency not to ($p = 0.8236$)—that is, H-CMARL-DE matches the static energy-aware planner on harvesting while operating dynamically. An ablation on the energy weight (Table V) motivates the choice $\eta = 3$: low-to-moderate values keep path-length efficiency near baseline while improving harvesting, whereas large values (5, 10) inflate both metrics and their variance. A step-size ablation similarly showed success falling from 99% to 95% to 90% as the per-step cell size shrank from 1 to 0.5 to 0.25, which informs the physical cell size (Section IV-D).

C. Physical testbed: devices, technologies, specifications

Figure 2 shows the testbed as the experimenter sees it: the overhead camera view with the projected grid, the detected markers, and the planner-facing minimap. This one frame captures the full perception story of Section III-D — the arena geometry is defined entirely by four corner markers, the robot is tracked purely externally, and the obstacle and energy grids consumed by the planners are built live from this view rather than supplied by a simulator. All campaign runs, including every figure in Section IV-E, were driven and logged through this interface.

Table VII summarises the testbed and Table VIII the ROS nodes. The robot is a TurtleBot 3 Burger [16] on ROS 2 Humble, with onboard computation on a Raspberry Pi 4B and motor control on an OpenCR board. Harvesting uses a small

TABLE VI

ROS/GAZEBO CASE STUDY [1]: CLASSICAL ROS-OMPL VS. CTDE VS. H-CMARL-DE, OVER TEN TRIALS. LAYOUT 1 IS A WAREHOUSE WITH AN INDUCTION-CHARGING LOOP; LAYOUT 2 USES SIX ILLUMINATION SOURCES WITH SOLAR HARVESTING. ALL CONFIGURATIONS REACHED 100% SUCCESS.

Method	Robots / Layout	Avg. reward	EH eff.	PL eff.	Obstacles	MCT (s)
ROS-OMPL	2 / 1	—	0.12 ± 0.03	1.110 ± 0.09	59	51.50 ± 5.8
H-CMARL-DE	2 / 1	68.65 ± 6.5	0.25 ± 0.05	1.17 ± 0.12	59	56.20 ± 5.7
CTDE	2 / 1	64.65 ± 7.1	0.21 ± 0.07	1.19 ± 0.14	59	58.50 ± 7.3
ROS-OMPL	4 / 2	—	0.13 ± 0.02	1.08 ± 0.09	48	47.12 ± 4.1
H-CMARL-DE	4 / 2	61.65 ± 5.4	0.29 ± 0.09	1.13 ± 0.08	48	51.10 ± 5.3
CTDE	4 / 2	58.65 ± 6.5	0.24 ± 0.08	1.17 ± 0.12	48	54.70 ± 5.6

TABLE VII
PHYSICAL TESTBED DEVICES AND TECHNOLOGIES.

Item	Specification
Robot	TurtleBot 3 Burger (≈ 14 cm wide), ROS 2 Humble; max ≈ 0.22 m/s, ≈ 2.84 rad/s
Onboard compute	Raspberry Pi 4B, I ² C bus /dev/i2c-1
Base controller	OpenCR (motors, IMU, servo); motor PWM ≈ 16 kHz
Harvesting	small solar panel $\rightarrow$ charge board $\rightarrow$ larger-than-stock battery
Power sensing	$3 \times$ INA219, 0.1Ω shunt, 100 Hz; channels solar / SBC / OpenCR
Stability signal	<i>undervoltage</i> under motor load (not thermal throttling); audited mid-campaign
Camera	overhead iPhone 8, 4K MJPEG stream
Localization	planar homography (position, marker centre) + planar-square PnP (heading); heuristic intrinsics
Markers	ArUco DICT_4X4_50; 4 corners + 1 robot marker
Arena grid	10×10 (15 cm cells) and 15×15 (13 cm cells); cells <i>smaller</i> than the robot
Navigation	rotate-then-drive, 20 Hz; IMU heading; multi-hop calibration (angular 1.0; linear surface-dependent)
Policy artifact	AIPPOm10EH_continued (SB3/PyTorch) $\rightarrow$ ONNX

TABLE VIII
THE ROS 2 NODES. THE ORCHESTRATOR REACHES ALL OF THEM ONLY THROUGH THE BRIDGE; PLANNER CHOICE IS A LAUNCH-TIME SELECTION. THE WHOLE STACK RELOCATES BETWEEN THE ROBOT PI (DECENTRALIZED) AND A DESKTOP VM (CENTRALIZED).

Node	Role	Runs on
bridge	TCP server, orchestrator gateway (:9090)	Pi / VM
navigation	drive-to-cell, rotate-then-drive	Pi / VM
PPO planner	learned policy (ONNX/torch)	Pi / VM
A* planner	energy-aware <i>or</i> shortest (launch variant)	Pi / VM
power/thermal	$3 \times$ INA219, undervoltage	robot Pi

solar panel feeding a larger-than-stock battery through a charge board. Localization and energy mapping are external (Fig. 3): ArUco markers fix the grid and recover pose, and a per-cell luminance analyser produces the energy map.

Calibration: Two motion-calibration factors were needed. The *angular* factor was 1.0 on both carpet and wood. In testing the IMU handled rotation essentially perfectly

and surface-independently, while the *linear* factor changed with surface and had to be redone for the wood arena. A single-meter calibration, which captures mostly steady-state cruising, mismatched the experiment’s short, burst-like cell-to-cell motion; a multi-hop calibration reproducing the actual movement profile was therefore used.

D. Experimental design and campaign

The campaign ran on two indoor maps: a 10×10 grid (15 cm cells) and a 15×15 grid (13 cm cells, ≈ 200 cm square, at the camera’s width limit). For each map we ran all six *compute-placement* $\times$ *planner* modes (centralized / decentralized $\times$ PPO / energy-A* / shortest-A*), keeping at least ten runs per mode. Goals and start cells were held constant across planners so conditions are directly comparable.

Goal-assist (PPO only): Using the custom rollout visualizer ahead of time revealed that the PPO policy sometimes required upwards of 80+ steps in these small environments. We can potentially attribute this to the stacking of 10 robots in-order to use the policy. To make it feasible on one battery charge, an opt-in, PPO-only “goal-assist” bounds run length: when the policy comes within one cell of the goal, the orchestrator still records the action it *wanted* to take but drives the final step itself and flags the run *goal_assisted*. We therefore report PPO success two ways, genuine and assisted.

Failure criterion: A stochastic policy can sample blocked moves; three consecutive illegal moves end the run as a failure. Both criteria are logged so PPO outcomes are objective and reproducible.

Metrics and statistics: Each run logs outcome, the executed path (length, step count, and added after we saw how much it mattered *turn count* / *total rotation*), time-to-goal, per-channel power at 100 Hz, planner inference time, and the end-to-end predict round trip. Net energy is harvested minus consumed. Within a map, outcome metrics (time, path, turns, energy) are computed over successful runs, mirroring the MCT/APL definitions above.

Data quality: Five runs aborted on tracker crashes (decentralized A*) and are counted separately; the decentralized-PPO session on the 15×15 map showed anomalous ~ 110 ms round trips and is excluded from latency-sensitive claims.



Fig. 2. The tracker during live operation on the 10×10 (left) and 15×15 (right) arenas. Four corner ArUco markers define the floor plane, onto which the virtual grid is projected; obstacle cells are shaded red and the robot is localized by its onboard marker (cell and heading overlaid). The top-right inset shows the planner-facing minimap - the obstacle/energy grids exactly as the active planner observes them - and the text overlay reports live pose, FPS, and run state. The 15×15 arena (right) spans ≈ 2 m at the camera's field-of-view limit, with the 3 lamps that generate the luminance field visible in frame.



Fig. 3. Online energy mapping. The raw overhead frame (left) is perspective-warped to the floor grid defined by the corner markers (center); a per-cell luminance analyzer then normalizes relative brightness to $[0, 1]$ to produce the energy map the planners consume (right; the robot marker is visible). Luminance is a proxy for harvestable energy.

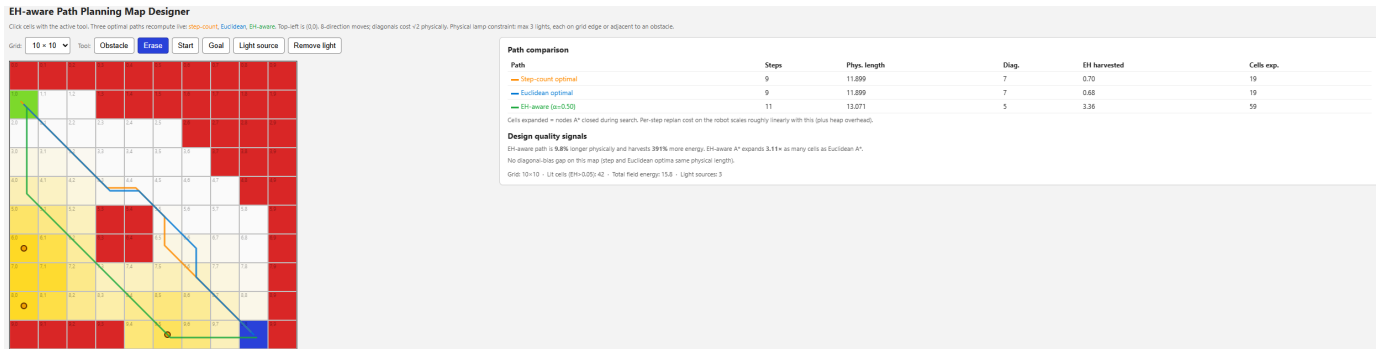


Fig. 4. Map-design and planner-comparison tool. Shortest-count, Euclidean, and energy-aware ($\alpha = 0.5$) A* paths recompute live; the energy-aware path is 9.8% longer yet harvests 391% more (simulated) energy. This tool and a PPO rollout visualizer were built to engineer maps that were scientifically interesting and worked under the egocentric and myopic shortcomings of the policy used.

E. Physical results

All findings below are group means over the two completed campaigns (the 10×10 and 15×15 maps, six mode $\times$ planner cells each); Table IX collects the headline numbers.

Net energy is a locomotion story: A run harvested only 0.003-0.005 mWh while consuming 9-20 mWh - three orders of magnitude apart. So net energy balance is essentially $-(\text{locomotion} + \text{compute})$ and net energy scales with run duration in every group (Figs. 11, 12). The panel is far too

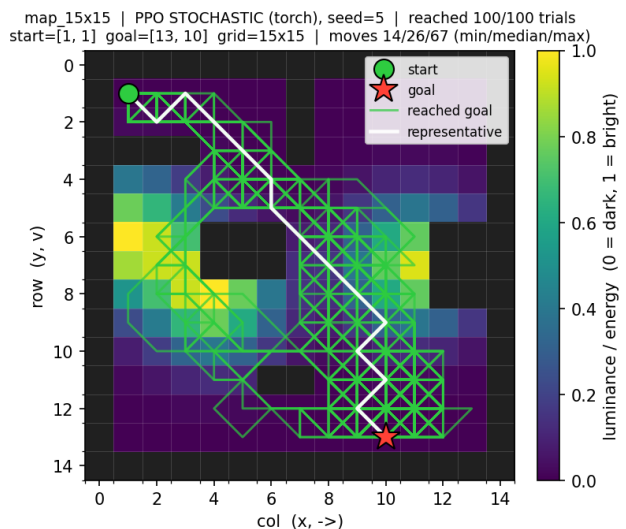
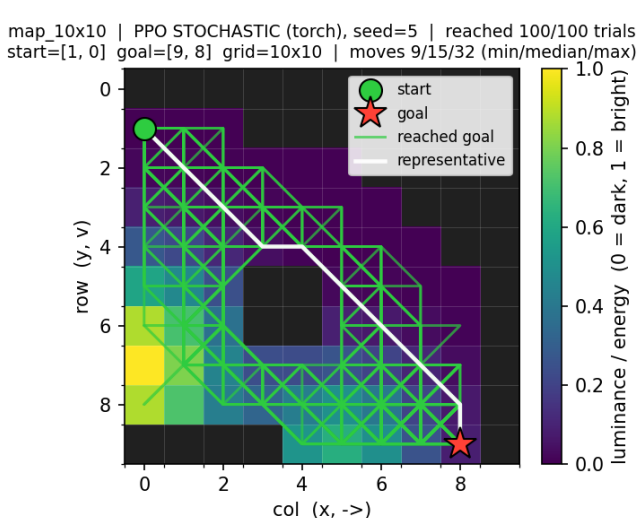


Fig. 5. Design-time PPO stochastic rollouts on the 10×10 (left) and 15×15 (right) maps over the luminance field. These vetted candidate maps against the policy before any hardware run; the on-robot behavior differed likely due to differences in the energy map of the physical arenas. (Section IV-E).

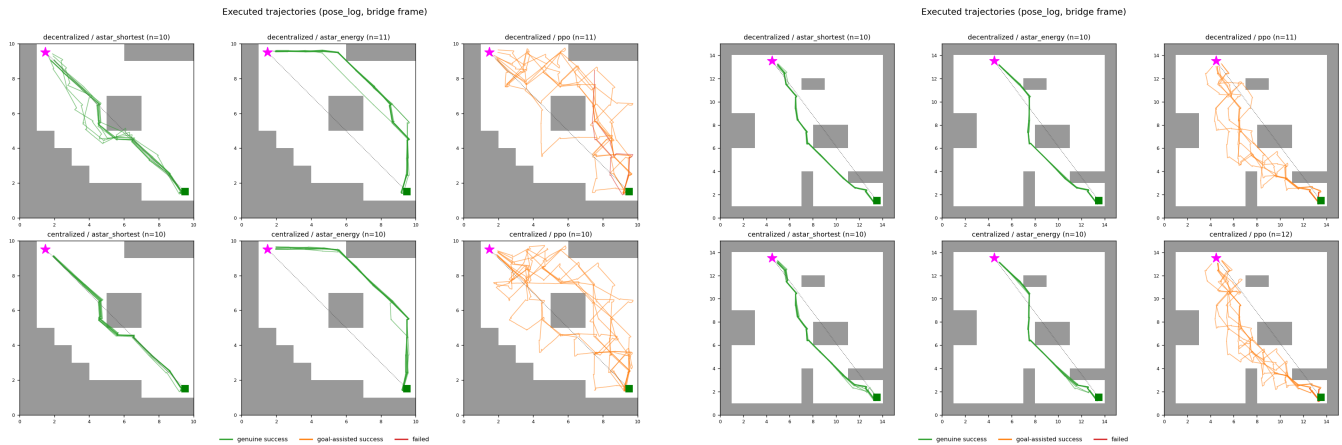


Fig. 6. Executed trajectories for all 6 modes, on both maps.

small for an *absolute* energy-positivity claim; the defensible comparison is relative.

Navigation: A dominates, PPO never closes the last cell:* Across 44 PPO runs there were *zero* genuine goal completions: all 33 PPO “successes” were goal-assisted (it is possible that goal-assisted trials would’ve finished on their own). The policy reliably wandered to within one cell of the goal and then failed to step onto it (note this is not a visibility problem, at one cell the goal is inside the window), giving a path efficiency of ≈ 0.6 – 0.7 against ≈ 0.84 – 0.88 for shortest-A* (Fig. 7). PPO behavior was equally poor on both maps (zero genuine successes, path efficiency 0.60 – 0.70), with successful runs taking 96 – 112 s versus 50 – 73 s for A*. (Figs. 13, 14).

Turn count, not distance, drives net energy: The net-energy winner *flipped* between maps. On the 10×10 map shortest-A* beat energy-A* (by ≈ 2.4 mWh centralized), simply by taking fewer steps (9 vs 12). On the 15×15 map

the result reversed (energy-A* won by 0.63 – 0.80 mWh). Both planners took ≈ 13 steps, but shortest-A* made *8 in-place heading changes versus 4* for energy-A*, and each rotation costs time and motor energy with no progress, so shortest-A* was even ≈ 3.5 – 5 s slower. This confirms the rotation-controller confound and shows it is not PPO-specific; we therefore report turn count alongside energy (Figs. 8, 9).

Energy-aware planning altered routes, but harvesting gains are unmeasurable: The harvested-energy difference points in the expected direction (0.0042 vs 0.0033 mWh centralized) but is not statistically significant and is dwarfed by consumption differences; with an undersized panel, an orientation-blind energy model, the electrical signal cannot confirm or refute the steering effect. We therefore support the steering claim geometrically (Fig. 6) rather than electrically.

Inference: A grows with the map, PPO stays flat:* As the map roughly doubled ($100 \rightarrow 225$ cells) every A* inference

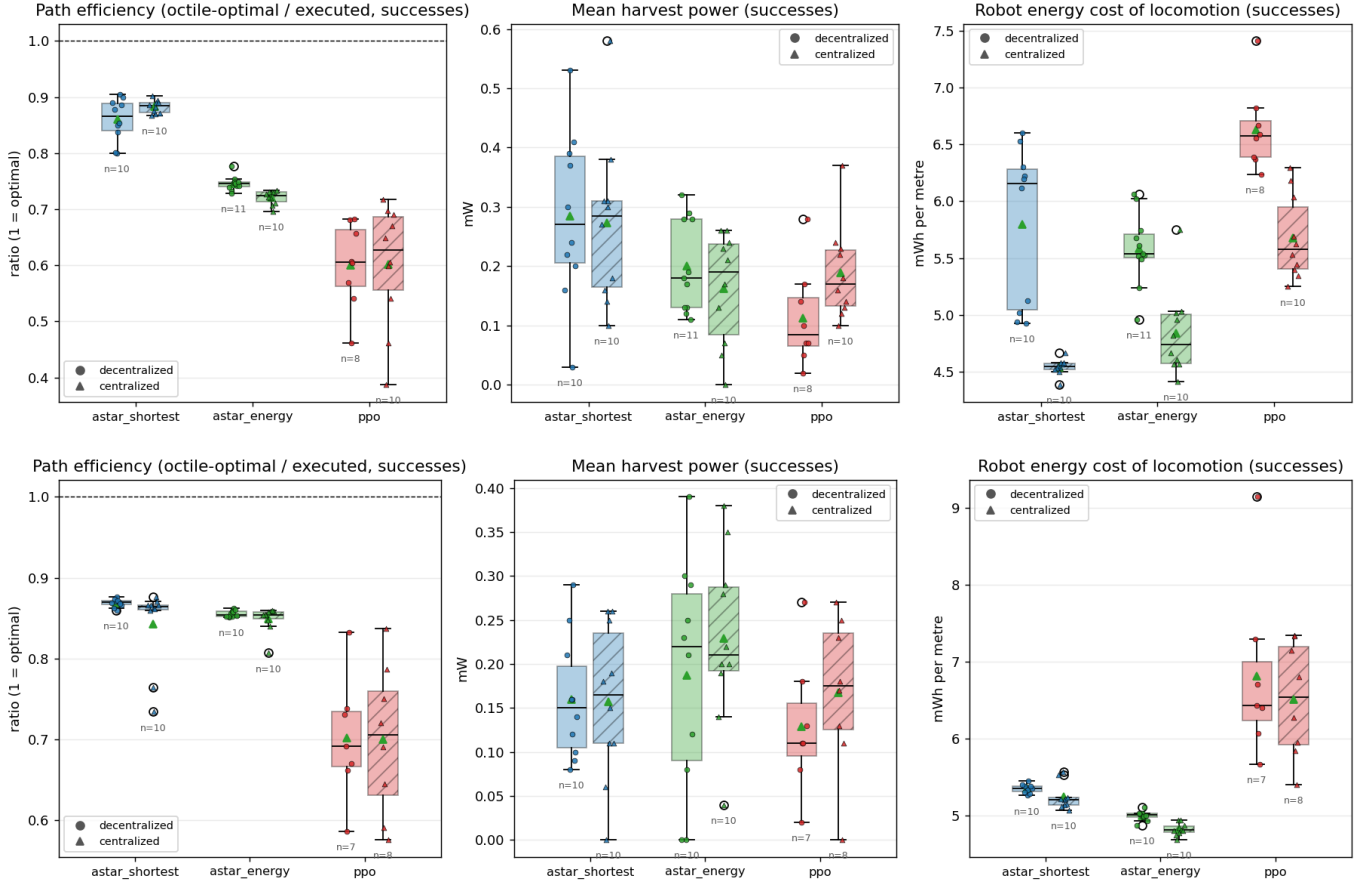


Fig. 7. Efficiency metrics per (mode, planner), successful runs only: 10×10 map (top) and 15×15 map (bottom). Left: path efficiency (octile-optimal over executed length; 1 = optimal) - shortest-A* reaches 0.84–0.88, energy-A* matches it on the 15×15 map where step counts are equal, and PPO trails at 0.60–0.70. Center: mean harvest power, far below locomotion power in every cell. Right: locomotion energy cost per meter, where PPO’s turn-heavy execution is the most expensive.

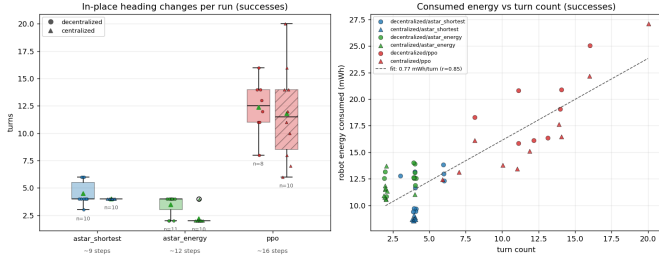


Fig. 8. Turn counts and consumed energy on the 10×10 map: here the step count difference (9 vs 12) dominates and shortest-A* wins despite making more turns.

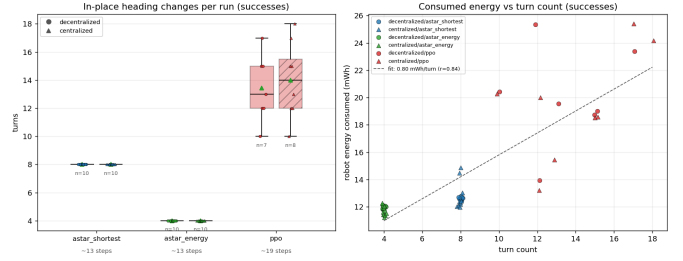


Fig. 9. Turn counts and consumed energy on the 15×15 map: with equal step counts (13), turn count (8 vs 4, deterministic per planner) decides the comparison and energy-A* wins.

mean grew $\sim 1.5\text{--}2\times$ (energy-A* on the Pi $1.40 \rightarrow 2.78$ ms; on the desktop $0.36 \rightarrow 0.70$ ms) while the PPO forward pass stayed flat (~ 2.7 ms Pi, ~ 0.5 ms desktop); on the larger map energy-A*’s mean caught PPO and its p99 crossed above it (Fig. 10). This is the clearest empirical support for the neural-planner amortization argument in the dataset. With the caveat that in our experiment PPO lost every navigation metric. Two more caveats: only two map sizes (so “consistent with

linear scaling in cell count,” not a fitted law), and inference is $\leq 5\%$ of the ~ 70 ms predict round trip, so end-to-end latency is network-bound at this scale - the crossover matters for compute energy and much larger maps, not today’s loop rate.

Compute placement: Centralization did not hurt the robot’s energy budget and cut SBC inference cost $\sim 4\times$. A larger centralized advantage on the 10×10 map ($+3.2$ mWh, ~ 17 s faster) did *not* replicate on the 15×15 map and is likely

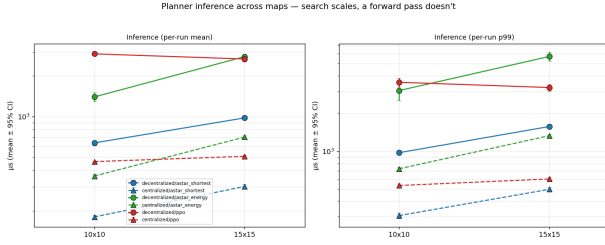


Fig. 10. Inference cost versus map size (experiment carried out over multiple days to fully charge battery). The A* search time grows with the cell count while the PPO forward pass is flat; on the 15×15 map A* catches PPO on the Pi (and its p99 crosses above).

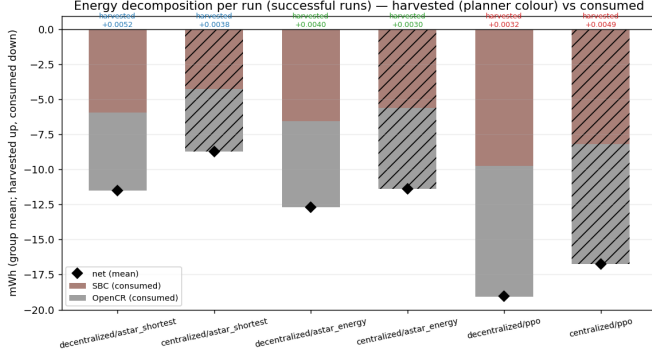


Fig. 11. Per-run energy decomposition on the 10×10 map (group means; harvested annotated above the axis, consumption stacked below). Harvested energy is ~ 3 orders of magnitude below consumption, so net energy is decided on the consumption side.

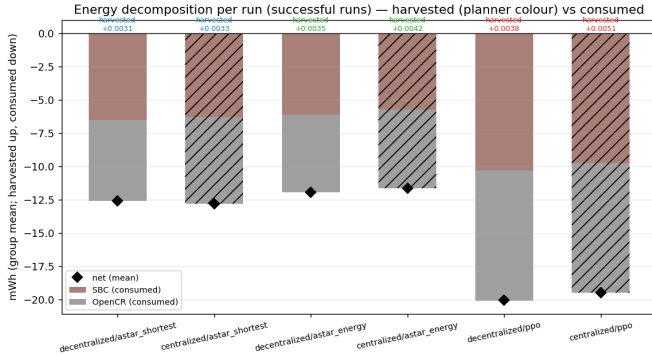


Fig. 12. Per-run energy decomposition on the 15×15 map (group means; harvested annotated above the axis, consumption stacked below). Harvested energy is ~ 3 orders of magnitude below consumption, so net energy is decided on the consumption side.

confounded (that session ran late with SBC throttling and control-loop overruns); we present it as unreplicated. Because planned RAPL passthrough failed, the desktop-side energy cost of centralization is unmeasured.

F. Threats to validity

Harvesting hardware and model: The on-board panel is undersized, so absolute net-energy figures cannot support a self-sufficiency claim. Real harvest is strongly orientation-

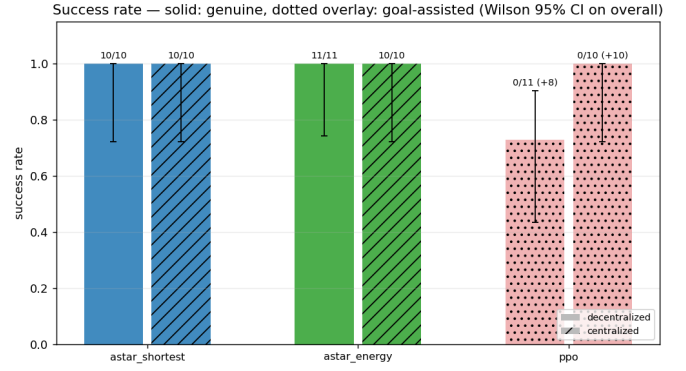


Fig. 13. (10×10) Success rates split into genuine versus goal-assisted. PPO's genuine successes are zero on both maps; A* succeeds genuinely.

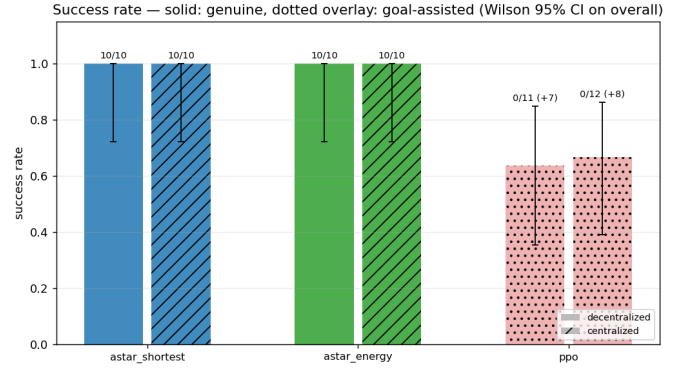


Fig. 14. (15×15) Success rates split into genuine versus goal-assisted. PPO's genuine successes are zero on both maps; A* succeeds genuinely.

dependent, yet both planners treat energy as an orientation-blind scalar field $E(x, y)$, so the policy optimizes *where* to be, never *which way to face*: a structural sim-to-real state-space mismatch.

Motion-layer confounds: The rotation controller is not fully turn-optimal, so turn-heavy paths pay an energy and time penalty unrelated to planning quality. An effect large enough to decide the A*-versus-A* comparison. Because cells are smaller than the robot, every move is an accelerate-settle-decelerate cycle, so consumed energy tracks (#cells, #turns) rather than physical distance. Occasional wheel slip on floor imperfections added run-to-run variance independent of the planner (tracking, notably, stayed valid throughout, which validates the external-localization choice).

PPO deployment: A ten-agent policy is driven as a single agent by stacking nine “ghost” robots on the real robot’s cell. This is an out-of-distribution input the policy never saw in training. Combined with the egocentric, goal-clamped 10×10 observation, which cannot plan around far or occluded obstacles, this is the leading explanation for the zero genuine successes. PPO’s arm should be read as the OOD single-agent slice, not the policy as designed.

Static-only regime: The campaign is fully static, so the policy’s claimed advantage in dynamic and stochastic environ-

TABLE IX

MEASURED OUTCOMES PER (MAP, MODE, PLANNER). SUCCESS COUNTS GENUINE (GOAL REACHED UNAIDED); PPO'S ASSISTED FINISHES IN PARENTHESES. TIME, PATH, TURNS AND NET ENERGY ARE MEAN $\pm$ SD OVER *successful* RUNS (INCL. ASSISTED); INFERENCE IS THE MEAN OVER ALL KEPT RUNS.

Map	Mode / planner	n	Success	Time (s)	Path (m)	Turns	Net energy (mWh)	Inference (ms)
10×10	dec. / shortest-A*	10	10/10	63.8 ± 10.0	1.98 ± 0.09	4.5 ± 1.1	-11.5 ± 1.8	0.64
	dec. / energy-A*	11	11/11	70.6 ± 5.0	2.28 ± 0.04	3.5 ± 0.9	-12.7 ± 0.9	1.40
	dec. / PPO	11	0/11 (+8)	105.5 ± 17.5	2.87 ± 0.40	12.4 ± 2.4	-19.1 ± 3.1	2.91
	cen. / shortest-A*	10	10/10	49.9 ± 0.9	1.92 ± 0.03	4.0 ± 0.0	-8.7 ± 0.2	0.18
	cen. / energy-A*	10	10/10	65.8 ± 5.6	2.35 ± 0.04	2.2 ± 0.6	-11.4 ± 0.9	0.36
	cen. / PPO	10	0/10 (+10)	95.6 ± 26.2	2.92 ± 0.65	11.8 ± 4.3	-16.7 ± 4.6	0.46
15×15	dec. / shortest-A*	10	10/10	69.3 ± 0.6	2.35 ± 0.01	8.0 ± 0.0	-12.6 ± 0.1	0.98
	dec. / energy-A*	10	10/10	65.8 ± 0.7	2.39 ± 0.01	4.0 ± 0.0	-11.9 ± 0.1	2.78
	dec. / PPO	11	0/11 (+7)	111.6 ± 21.0	2.94 ± 0.32	13.4 ± 2.4	-20.0 ± 3.6	2.67
	cen. / shortest-A*	10	10/10	73.4 ± 6.0	2.43 ± 0.16	8.0 ± 0.0	-12.8 ± 1.1	0.30
	cen. / energy-A*	10	10/10	66.4 ± 2.2	2.41 ± 0.05	4.0 ± 0.0	-11.6 ± 0.3	0.70
	cen. / PPO	12	0/12 (+8)	111.1 ± 23.5	2.97 ± 0.40	14.0 ± 2.7	-19.4 ± 4.1	0.51

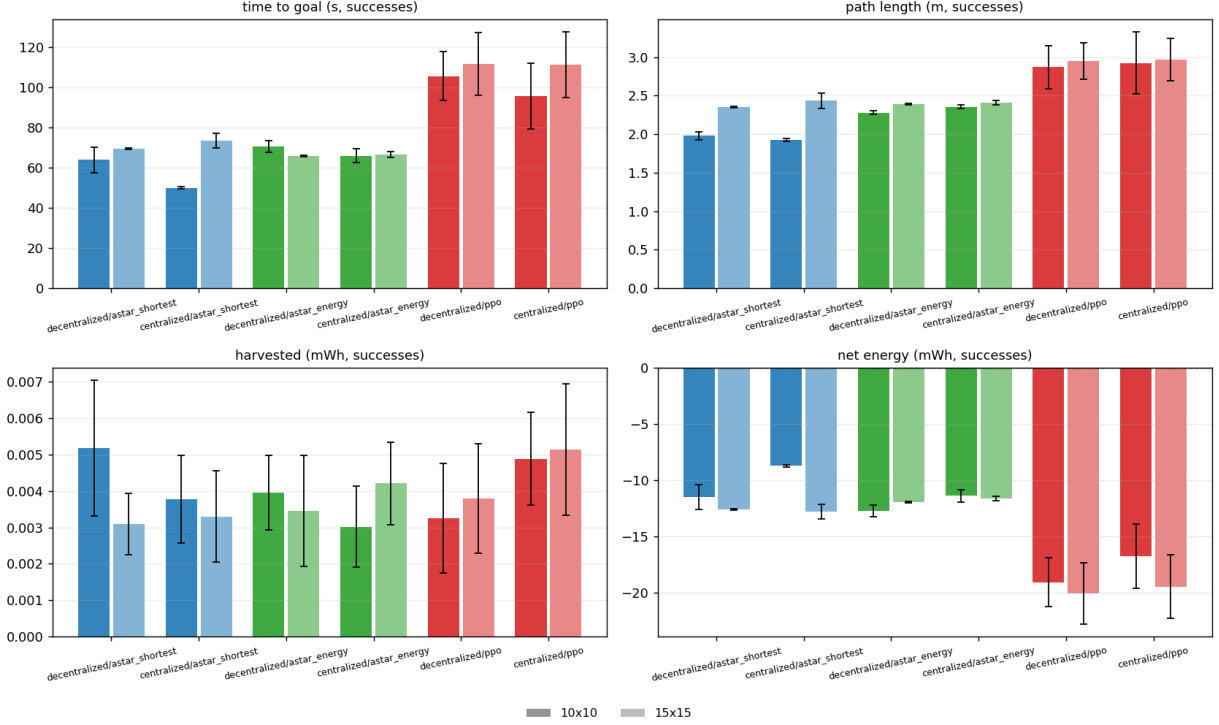
Group means $\pm$ 95% CI per map (successful runs)

Fig. 15. Group means $\pm$ 95% CI per map for the four headline outcome metrics (successful runs only). Bar shading distinguishes the 10×10 (dark) and 15×15 (light) maps; planner colours as elsewhere. Note the PPO columns: successful PPO runs are the slowest and longest in every configuration, and net energy is correspondingly the most negative.

ments (Table I) is never exercised; PPO is tested only where classical A* is provably optimal.

Perception: Overhead fiducial localization is occlusion-fragile, a wire crossing the arena dropped tracking, and shares the visible spectrum with the lights, so low, bright lamps can over-expose the marker. The luminance energy map is a flawed proxy for the energy map, compressed to indoor dynamic ranges. It is not calibrated to harvestable joules, A cell which received a value of 0.8 in the energy map might

not be sufficiently bright to actually activate the solar panel.

Instrumentation and embedding: Undervoltage under motor load (not heat) is the stability limiter; degraded-battery runs were flagged and audited out. The Raspberry Pi 4B is more power-hungry than the swarm embedded vision the method targets, so the rig under-represents the embedded-efficiency upside, and desktop compute energy is unmeasured.

V. CONCLUSION

The source paper showed in simulation that energy-harvesting-aware planning—and H-CMARL-DE in particular—can raise harvesting efficiency by up to 240% over non-EH-aware planning while keeping paths near-optimal and inference real-time [1], and it named real-world hardware validation as future work. This report ran that validation on a physical TurtleBot 3 across two indoor maps and two compute placements; the picture is honest and instructive.

At indoor scale, harvested energy was three orders of magnitude below consumption, so the experiment is a *relative* comparison, not a demonstration of energy self-sufficiency. Net energy proved to be a locomotion story dominated by turn count and per-cell stop-start motion. So much so that turn count, not harvesting, decided the energy-aware-versus-shortest A* comparison and even flipped it between maps. Energy-aware planning consistently altered the chosen route, but the harvesting payoff was below measurement noise given the undersized panel, in-door setting and an orientation-blind energy model.

The clearest pro-learning result was computational: A* search time grew with map size while the PPO forward pass stayed flat, and on the Pi, the two crossed on the larger map, empirical support for the neural-planner amortization argument, albeit only one for compute energy and much larger maps, since end-to-end latency here is network-bound. Against this, the learned policy navigated poorly on hardware: it never once closed the final step to a goal unaided across 44 runs, at roughly 0.6 – 0.7 path efficiency, behavior we attribute to its egocentric 10×10 observation and to running a ten-agent policy as a single out-of-distribution agent. As deployed, then, the classical energy-aware A* is the better physical navigator, and the learned planner’s advantage is confined to inference scaling.

These conclusions are bounded by real limitations (Section IV-F): an undersized panel and orientation-blind energy model, a turn-cost and stop-start motion confound, occlusion- and exposure-fragile overhead localization, under-voltage under load, and a fully static arena that never exercises the policy’s intended dynamic-environment strengths. The natural future work follows directly: add an orientation/heading channel and calibrate luminance to harvestable energy; use cells at least robot-sized with multi-cell commitment (the paper’s commit window) to remove the stop-start tax; retrain or adapt the policy with a larger or global observation and a true goal channel to cure the myopia; and inject virtual moving obstacles and time-varying light—which the orchestrator already builds per step—to finally test the adaptivity that motivated a learned planner in the first place.

ACKNOWLEDGEMENT

The author thanks Prof. dr. J. Famaey as organizer and M. Mokhtari for their supervision and for providing the trained policy and the simulation study this work is based on. For helping with the modified hardware of the TurtleBot3 the author also thanks D. Subotic.

REFERENCES

- [1] M. Mokhtari, B. Vanderborght, and J. Famaey, “Energy harvesting aware path planning for ambiently-powered multi-robot systems,” *Robotics and Autonomous Systems*, vol. 197, p. 105260, 2026.
- [2] P. A. Plonski, P. Tokekar, and V. Isler, “Energy-efficient path planning for solar-powered mobile robots,” *Journal of Field Robotics*, vol. 30, no. 4, pp. 583–601, 2013.
- [3] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [4] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [5] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [7] M. Mokhtari, B. Vanderborght, and J. Famaey, “Energy-aware multi-robot task scheduling using meta-heuristic optimization methods for ambiently-powered robot swarms,” *Robotics and Autonomous Systems*, vol. 186, p. 104898, 2025.
- [8] ONNX Runtime developers, “ONNX Runtime,” <https://onnxruntime.ai>, 2021, cross-platform inference and training accelerator.
- [9] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dornmann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [10] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, “Automatic generation and detection of highly reliable fiducial markers under occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.
- [11] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *Proc. 33rd Int. Conf. on Machine Learning (ICML)*, ser. PMLR, vol. 48, 2016, pp. 1928–1937.
- [12] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024.
- [13] S. Das and P. N. Suganthan, “Differential evolution: A survey of the state-of-the-art,” *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 1, pp. 4–31, 2011.
- [14] I. A. Şucan, M. Moll, and L. E. Kavraki, “The open motion planning library,” *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, 2012.
- [15] W. Hess, D. Kohler, H. Rapp, and D. Andor, “Real-time loop closure in 2D LIDAR SLAM,” in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2016, pp. 1271–1278.
- [16] R. Amsters and P. Slaets, “Turtlebot 3 as a robotics education platform,” in *Robotics in Education (RIE 2019)*, ser. Advances in Intelligent Systems and Computing, vol. 1023. Springer, 2020, pp. 170–181.